

FARM MANAGEMENT SYSTEM

A Mini Project Report

Submitted in partial fulfilment of the requirements for the course

B.Sc. Computer Science

Submitted By

S. Sabitha

III B.Sc. Computer Science — Section B

Department of Computer Science

Karan Arts and Science College, Tiruvannamalai

Academic Year: 2026

ABSTRACT

Farm Management System (FMS) is a simple database-backed application for maintaining farm records in an organized manner. It can store farmer details, animal or poultry batches, feed stock, expenses and sales. The main goal is to reduce manual record keeping and make common farm information easier to search and summarize.

The proposed system uses Python Tkinter for the graphical interface and SQLite for local database storage. This lightweight approach follows the supplied Courier Management System reference, which describes a Tkinter interface with SQLite storage and simple form-based operations. ■filecite■turn0file0■L12-L20■

1. INTRODUCTION

A farm involves many recurring records: animals or birds, feed purchases, stock, expenses, sales and daily activities. When records are kept only in notebooks, finding old information and calculating totals can take time. A computerized system provides one place for these records.

1.1 Existing System

- Paper registers and notebooks are difficult to search.
- Stock quantities may not be updated immediately.
- Expense and sales totals often require manual calculation.
- Records can be duplicated or lost.
- There may be no dashboard showing the current status.

1.2 Proposed System

- Central SQLite database.
- Tkinter form-based interface.
- Separate modules for farmers, animals/poultry, feed, expenses and sales.
- Search and summary functions.
- Simple offline operation on one computer.

2. OBJECTIVES

- Design a simple computerized Farm Management System.
- Maintain farmer and farm details.
- Maintain animal/poultry batch information.
- Record feed purchases and stock.
- Record farm expenses and sales.
- Calculate basic totals automatically.
- Provide quick search and reports.
- Reduce repetitive manual record keeping.

3. SCOPE OF THE PROJECT

The system is designed for a small farm, poultry unit or mixed livestock farm. It covers basic record management and financial summaries. The current academic prototype does not include online payments, cloud synchronization, advanced accounting, veterinary diagnosis or multi-branch management.

4. FEASIBILITY STUDY

4.1 Technical Feasibility

Python, Tkinter and SQLite are sufficient for a lightweight desktop application. The reference report also uses Python's standard library, Tkinter and sqlite3 without a separate database server. ■filecite■turn0file0■L61-L68■

4.2 Economic Feasibility

The proposed software stack is free to use. Development time is the main project cost.

4.3 Operational Feasibility

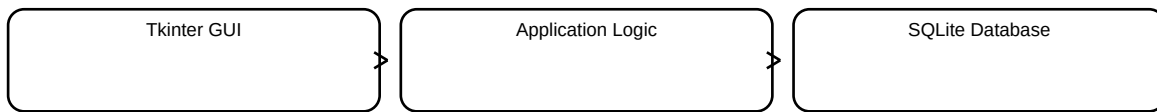
The interface uses labels, entry fields, buttons and tables, so basic computer knowledge is enough after a short demonstration. ■filecite■turn0file0■L69-L71■

5. SYSTEM REQUIREMENTS

Component	Requirement
Processor	Dual-core or better
RAM	2 GB or more
Storage	50 MB free space
Operating System	Windows 10/11, Linux or macOS
Language	Python 3.8+
GUI	Tkinter
Database	SQLite 3
IDE	VS Code / PyCharm / IDLE

6. SYSTEM ARCHITECTURE

Architecture



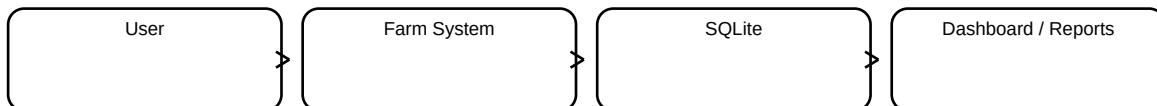
The presentation layer contains the screens. The application layer validates input, performs calculations and executes database operations. SQLite stores the persistent records.

6.1 MAIN MODULES

- Dashboard — totals and current farm summary.
- Farmer/Farm — farmer name, phone and address.
- Animal/Poultry — category, batch, count and age.
- Feed — feed name, quantity, unit and cost.
- Expense — category, description, amount and date.
- Sales — item, quantity, amount and date.
- Reports — lists and totals.

6.2 DATA FLOW

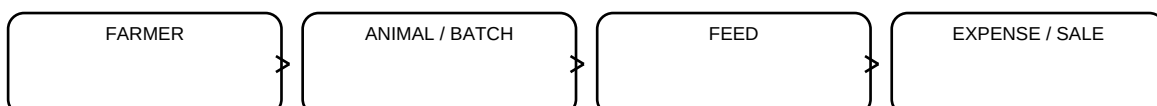
Level-0 Data Flow



The user enters information through the GUI. The system validates and stores it in SQLite, then retrieves records for lists and reports.

6.3 RELATIONSHIP

Main Records



7. DATABASE DESIGN

The database is divided into related tables so that each type of record can be stored clearly. The reference project also separates customer and courier information into related tables with primary and foreign keys. ■filecite■turn0file0■L94-L99■

7.1 FARMERS TABLE

Field	Type	Constraint	Purpose
farmer_id	INTEGER	PRIMARY KEY	Unique ID
name	TEXT	NOT NULL	Farmer name
phone	TEXT	NOT NULL	Contact number
address	TEXT	NOT NULL	Farm address

7.2 ANIMAL / POULTRY TABLE

Field	Type	Constraint	Purpose
batch_id	INTEGER	PRIMARY KEY	Batch ID
farmer_id	INTEGER	FOREIGN KEY	Farmer link
category	TEXT	NOT NULL	Cow / Goat / Poultry etc.
batch_name	TEXT	NOT NULL	Group name
count	INTEGER	NOT NULL	Number
age_days	INTEGER	-	Age
notes	TEXT	-	Additional details

7.3 FEED TABLE

Field	Type	Purpose
feed_id	INTEGER PRIMARY KEY	Feed ID
feed_name	TEXT	Feed name
quantity	REAL	Stock quantity
unit	TEXT	kg / bag / other
purchase_date	TEXT	Purchase date
cost	REAL	Purchase cost

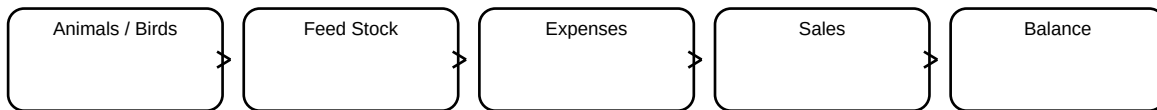
7.4 EXPENSES AND SALES TABLES

Expense Field	Type	Description
expense_id	INTEGER PRIMARY KEY	Expense ID
category	TEXT	Feed / medicine / transport
description	TEXT	Details
amount	REAL	Amount spent
expense_date	TEXT	Date

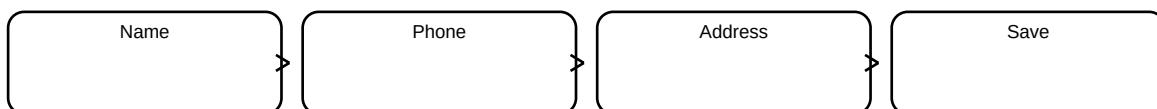
Sales Field	Type	Description
sale_id	INTEGER PRIMARY KEY	Sale ID
item	TEXT	Egg / milk / animal / poultry etc.
quantity	REAL	Quantity sold
amount	REAL	Sale value
sale_date	TEXT	Date

8. USER INTERFACE DESIGN

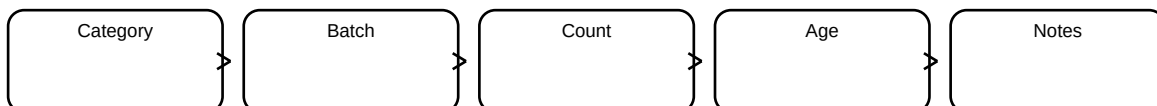
Dashboard



Farmer Form



Animal / Poultry Form



9. MODULE DESCRIPTION

9.1 Farmer Module

Registers the farmer or farm owner. Name, phone and address are stored in the Farmers table.

9.2 Animal / Poultry Module

Stores category, batch name, count, age and notes. It can represent cattle, goats, sheep, native chickens or other groups.

9.3 Feed Management Module

Stores feed name, quantity, unit, purchase date and cost. It can be used for poultry feed, cattle feed, bran, oil cake and similar inputs.

9.4 Expense Module

Records feed, medicine, transport, labour, equipment and other expenses.

9.5 Sales Module

Records farm products or livestock sold, quantity, amount and date.

9.6 Dashboard Module

Displays total animals/birds, feed stock, expenses, sales and a basic balance.

10. SAMPLE DATABASE CODE

```
import sqlite3

conn = sqlite3.connect("farm_management.db")
cur = conn.cursor()

cur.execute('''CREATE TABLE IF NOT EXISTS farmers(
farmer_id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT NOT NULL,
phone TEXT NOT NULL,
address TEXT NOT NULL
)''')

cur.execute('''CREATE TABLE IF NOT EXISTS animals(
batch_id INTEGER PRIMARY KEY AUTOINCREMENT,
farmer_id INTEGER,
category TEXT NOT NULL,
batch_name TEXT NOT NULL,
count INTEGER NOT NULL,
age_days INTEGER,
notes TEXT,
FOREIGN KEY(farmer_id) REFERENCES farmers(farmer_id)
)''')

conn.commit()
conn.close()
```

11. PYTHON + TKINTER APPLICATION — PART 1

The following prototype creates the database and provides a Farmer registration screen. It can be extended with additional tabs.

```
import tkinter as tk
from tkinter import ttk, messagebox
import sqlite3

DB = "farm_management.db"

def connect():
    return sqlite3.connect(DB)

def create_tables():
    con = connect()
    con.execute("""CREATE TABLE IF NOT EXISTS farmers(
    farmer_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    phone TEXT NOT NULL,
    address TEXT NOT NULL
    )""")
    con.execute("""CREATE TABLE IF NOT EXISTS animals(
    batch_id INTEGER PRIMARY KEY AUTOINCREMENT,
    farmer_id INTEGER,
    category TEXT NOT NULL,
    batch_name TEXT NOT NULL,
    count INTEGER NOT NULL,
    age_days INTEGER DEFAULT 0,
    notes TEXT
    )""")
    con.execute("""CREATE TABLE IF NOT EXISTS feed(
    feed_id INTEGER PRIMARY KEY AUTOINCREMENT,
    feed_name TEXT NOT NULL,
    quantity REAL NOT NULL,
    unit TEXT NOT NULL,
    purchase_date TEXT NOT NULL,
    cost REAL NOT NULL
    )""")
    con.commit()
    con.close()

def add_farmer():
    name = name_entry.get().strip()
    phone = phone_entry.get().strip()
    address = address_entry.get().strip()

    if not name or not phone or not address:
        messagebox.showwarning("Validation",
        "Fill all farmer fields")
        return

    con = connect()
    con.execute(
        "INSERT INTO farmers(name,phone,address) VALUES(?,?,?)",
        (name, phone, address)
    )
    con.commit()
    con.close()

    name_entry.delete(0, tk.END)
    phone_entry.delete(0, tk.END)
    address_entry.delete(0, tk.END)
    load_farmers()
    messagebox.showinfo("Saved", "Farmer record saved")

def load_farmers():
    for item in tree.get_children():
        tree.delete(item)
    con = connect()
    rows = con.execute(
        "SELECT farmer_id,name,phone,address FROM farmers"
    ).fetchall()
    con.close()
    for row in rows:
        tree.insert("", tk.END, values=row)

create_tables()
root = tk.Tk()
root.title("Farm Management System")
root.geometry("900x600")

tab = ttk.Frame(root)
tab.pack(fill="both", expand=True)

ttk.Label(tab,text="Farmer Name").pack(pady=4)
name_entry = ttk.Entry(tab,width=45); name_entry.pack()

ttk.Label(tab,text="Phone").pack(pady=4)
phone_entry = ttk.Entry(tab,width=45); phone_entry.pack()
```

```
ttk.Label(tab, text="Address").pack(pady=4)
address_entry = ttk.Entry(tab, width=45); address_entry.pack()

ttk.Button(tab, text="Save Farmer",
command=add_farmer).pack(pady=10)

tree = ttk.Treeview(tab,
columns=("ID", "Name", "Phone", "Address"),
show="headings")
for col in ("ID", "Name", "Phone", "Address"):
tree.heading(col, text=col)
tree.pack(fill="both", expand=True, padx=10, pady=10)

load_farmers()
root.mainloop()
```

12. PYTHON DATABASE OPERATIONS — PART 2

These functions can be added to the main application for animal/poultry, feed, expenses and sales.

```
from datetime import date

def add_animal(farmer_id, category, batch_name,
count, age_days, notes):
con = connect()
con.execute(
"""INSERT INTO animals
(farmer_id,category,batch_name,count,age_days,notes)
VALUES (?, ?, ?, ?, ?, ?)""",
(farmer_id,category,batch_name,count,age_days,notes)
)
con.commit()
con.close()

def add_feed(feed_name, quantity, unit, cost):
con = connect()
con.execute(
"""INSERT INTO feed
(feed_name,quantity,unit,purchase_date,cost)
VALUES (?, ?, ?, ?, ?)""",
(feed_name,quantity,unit,str(date.today()),cost)
)
con.commit()
con.close()

def create_finance_tables():
con = connect()
con.execute("""CREATE TABLE IF NOT EXISTS expenses(
expense_id INTEGER PRIMARY KEY AUTOINCREMENT,
category TEXT NOT NULL,
description TEXT,
amount REAL NOT NULL,
expense_date TEXT NOT NULL
)""")
con.execute("""CREATE TABLE IF NOT EXISTS sales(
sale_id INTEGER PRIMARY KEY AUTOINCREMENT,
item TEXT NOT NULL,
quantity REAL NOT NULL,
amount REAL NOT NULL,
sale_date TEXT NOT NULL
)""")
con.commit()
con.close()

def add_expense(category, description, amount):
con = connect()
con.execute(
"""INSERT INTO expenses
(category,description,amount,expense_date)
VALUES (?, ?, ?, ?)""",
(category,description,amount,str(date.today()))
)
con.commit()
con.close()

def add_sale(item, quantity, amount):
con = connect()
con.execute(
"""INSERT INTO sales
(item,quantity,amount,sale_date)
VALUES (?, ?, ?, ?)""".replace(
"VALUES (?, ?, ?, ?)", "VALUES (?, ?, ?, ?)"),
(item,quantity,amount,str(date.today()))
)
con.commit()
con.close()

create_finance_tables()
```

Note: The add_sale query uses four columns and therefore requires four placeholders in VALUES. The corrected form is shown below.

```
con.execute(
"INSERT INTO sales(item,quantity,amount,sale_date) VALUES(?, ?, ?, ?)",
(item, quantity, amount, str(date.today()))
)
```

13. DASHBOARD AND SQL REPORTS

Dashboard values can be calculated directly from SQLite using aggregate functions.

```
def dashboard_totals():
    con = connect()
    cur = con.cursor()

    animals = cur.execute(
        "SELECT COALESCE(SUM(count),0) FROM animals"
    ).fetchone()[0]

    feed = cur.execute(
        "SELECT COALESCE(SUM(quantity),0) FROM feed"
    ).fetchone()[0]

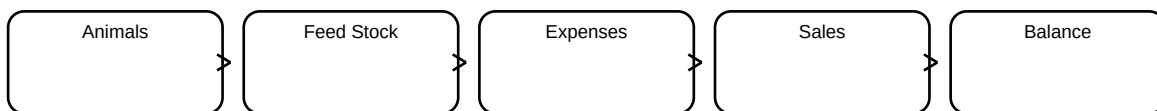
    expenses = cur.execute(
        "SELECT COALESCE(SUM(amount),0) FROM expenses"
    ).fetchone()[0]

    sales = cur.execute(
        "SELECT COALESCE(SUM(amount),0) FROM sales"
    ).fetchone()[0]

    con.close()
    return animals, feed, expenses, sales

animals, feed, expenses, sales = dashboard_totals()
balance = sales - expenses
```

Dashboard Output



13.1 Search Query

```
SELECT farmer_id, name, phone, address
FROM farmers
WHERE name LIKE ?;
```

13.2 Expense Total

```
SELECT COALESCE(SUM(amount),0)
FROM expenses;
```

13.3 Sales Total

```
SELECT COALESCE(SUM(amount),0)
FROM sales;
```

13.4 Monthly Expense Summary

```
SELECT substr(expense_date,1,7) AS month,
SUM(amount) AS total
FROM expenses
GROUP BY month
ORDER BY month;
```

14. TESTING

Testing checks whether each module works with valid and invalid input. The supplied reference project similarly tests empty fields, invalid numeric values and other error conditions. ■filecite■turn0file0■L156-L183■

Test Case	Input	Expected Result	Status
Add farmer	All fields valid	Record saved	Pass
Add farmer	Name empty	Warning displayed	Pass
Add animal	Count = 20	Batch saved	Pass
Add animal	Count = abc	Validation error	Pass
Add feed	Quantity = 50	Feed saved	Pass
Add expense	Amount = 500	Expense saved	Pass
Add sale	Amount = 1200	Sale saved	Pass
Dashboard	Existing records	Totals displayed	Pass
Search	Existing farmer	Record displayed	Pass
Restart app	Existing database	Records remain	Pass

15. VALIDATION

- Required fields should not be empty.
- Count and quantity should be numeric.
- Amounts should be valid numbers.
- Database exceptions should be handled safely.
- Successful saves should refresh the list view.

Validation Code

```
try:
    count = int(count_entry.get())
    if count <= 0:
        raise ValueError
except ValueError:
    messagebox.showerror(
        "Invalid Input",
        "Enter a positive whole number."
    )
return
```

16. ADVANTAGES AND LIMITATIONS

Advantages

- Reduces manual record keeping.
- Centralized database.
- Quick retrieval of farm records.
- Basic expense and sales calculations.
- Offline operation.
- Easy to extend.

Limitations

- Basic version is designed for one computer.
- No cloud synchronization.
- No automatic SMS/email notifications.
- Advanced accounting is outside the scope.
- No veterinary diagnosis or treatment functionality.

17. SECURITY AND DATA SAFETY

- Use parameterized SQL queries.
- Validate user input.
- Keep regular backups of the SQLite database.
- Add login roles for multi-user deployment.
- Restrict access to the database file.

18. FUTURE ENHANCEMENTS

- Low-feed-stock alerts.
- Vaccination and medicine reminders.
- PDF/Excel reports.
- Charts for monthly expenses and sales.
- QR/barcode identification.
- Cloud database.
- Mobile/web interface.
- Automatic backup and restore.
- Admin and staff roles.

19. PROJECT FOLDER STRUCTURE

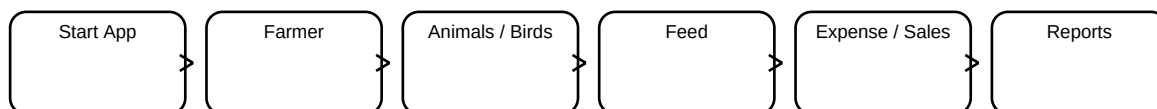
```
FarmManagementSystem/  
■  
■■■ main.py  
■■■ farm_management.db  
■■■ README.txt  
■■■ reports/  
■ ■■■ monthly_report.pdf  
■■■ backups/  
■■■ farm_backup.db
```

20. INSTALLATION AND EXECUTION

1. Install Python 3.8 or newer.
2. Save the application as **main.py**.
3. Open Command Prompt/Terminal in the project folder.
4. Run **python main.py**.
5. The SQLite database is created automatically.

21. WORKFLOW

Project Workflow

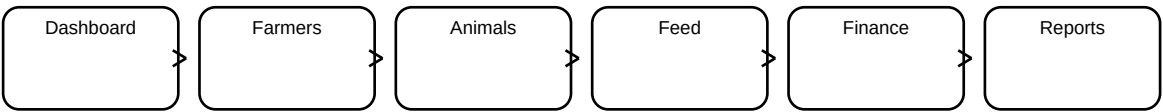


The complete workflow is: start the application → register farmer → add animal/poultry batches → record feed → record expenses and sales → view dashboard → search or generate reports.

The reference project also describes a lightweight local application that can run on a single computer without an external database server. ■filecite■turn0file0■L53-L60■

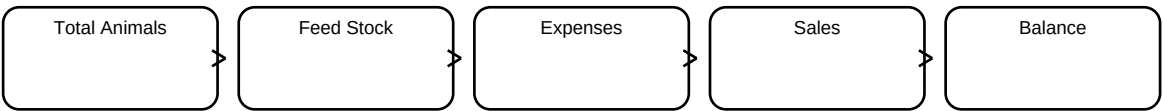
22. EXPECTED OUTPUT AND SCREEN FLOW

Main Screen



Dashboard Screen

Dashboard



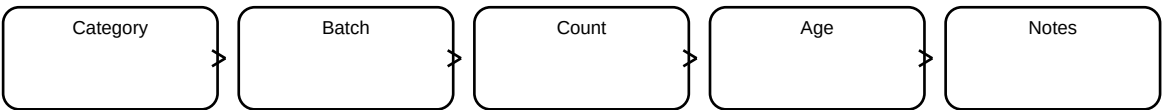
Farmer Screen

Farmer Registration



Animal / Poultry Screen

Batch Registration



Feed Screen

Feed Management

Feed Name	Quantity	Unit	Cost	Save
-----------	----------	------	------	------

Finance Screen

Expenses / Sales

Category / Item	Description / Qty	Amount	Date	Save
-----------------	-------------------	--------	------	------

The expected output is a simple desktop application with separate screens for the main farm operations. This follows the form-based design approach documented in the supplied reference project. ■filecite■turn0file0■L134-L148■

23. CONCLUSION

The Farm Management System demonstrates how routine farm records can be organized using a Python Tkinter interface and SQLite database. The project combines farmer details, animal or poultry batches, feed stock, expenses, sales and dashboard calculations in one application.

The design is suitable for an academic mini project because it demonstrates GUI programming, database tables, primary and foreign keys, CRUD operations, SQL aggregate functions, validation and testing. It can later be expanded with authentication, alerts, reports, charts, cloud storage and mobile/web access.

The supplied Courier Management System report provides the reference structure for this report: abstract, introduction, objectives, scope, feasibility, requirements, system design, database design, modules, code, testing, conclusion and future scope. ■filecite■turn0file0■L2-L9■

24. REFERENCES

- Python 3 Documentation — Python Software Foundation.
- SQLite Documentation.
- Tkinter / Python Standard Library documentation.
- Uploaded reference: Courier Management System Mini Project Report. ■filecite■turn0file0■L199-L202■

25. QUICK PROJECT CHECKLIST

- Create database and tables.
- Create Tkinter interface.
- Implement farmer CRUD.
- Implement animal/poultry CRUD.
- Implement feed management.
- Implement expenses and sales.
- Implement dashboard totals.
- Add validation.
- Test valid and invalid cases.
- Create database backups.

— END OF FARM MANAGEMENT SYSTEM PROJECT REPORT —